

```

%% scripts/run_experiments_v1.m

% V1 experiment runner with >=20 data points per analysis curve.
% Four-factor analysis + per-factor optimization + plots + savings.


clear; clc;


% ---- Ensure we are in project root (Simulation)
thisFile = mfilename('fullpath');
projRoot = fileparts(fileparts(thisFile)); % ../Simulation
cd(projRoot);


addpath(fullfile(projRoot,"scripts"));


% ---- Load nominal params into Base Workspace
P0 = params_nominal();
assignin('base','P',P0);


% ---- Model
mdl = "LHP_DT_v1";
mdlPath = fullfile(projRoot, "models", mdl + ".slx");
if ~isfile(mdlPath)
    error("Cannot find model file: %s", mdlPath);
end


load_system(mdlPath);
set_param(mdl, "StopTime", num2str(P0.t_cycle_s));

```

```

% ---- Output folders

ensureDir(fullfile(projRoot,"results","tables"));
ensureDir(fullfile(projRoot,"results","figs"));

% ===== Experiment grids: >=20 points per simulation curve =====

nPts = 20;

% Same physical ranges as the original script, but densified.
loads = linspace(0.0, 1.2, nPts); % no-load to overload
speeds = linspace(0.4, 1.0, nPts); % low to high speed ratio
temps = linspace(-20, 50, nPts); % low to high ambient temperature

% Continuous operator behaviour intensity:
% 0 = smooth/eco operation, 1 = aggressive operation.
modes = linspace(0.0, 1.0, nPts);

% ===== Baseline assumptions =====

BASE.fixedLoad = 1.0;
BASE.fixedSpeed = 0.7;
BASE.fixedTemp = 25;
BASE.fixedMode = 1; % aggressive by default

% ===== OPT knobs (V1 simplifications) =====

OPT = struct();
OPT.enforce_smooth_mode = true;

% Denser speed search so the optimum is less sensitive to discretisation.
OPT.search_speeds = linspace(min(speeds), max(speeds), max(41,nPts));

```

```
OPT.standby_aux_factor = 0.80;
```

```
OPT.low_load_threshold = 0.3;
```

```
OPT.temp_ctrl_extra_W = 1000;
```

```
OPT.k_eta_reduction = 0.4;
```

```
% ===== Run 4 analyses + optimizations =====
```

```
disp("Running Scenario 1: Load analysis (20 points)...");
```

```
S1_base = run_sweep(mdl, P0, loads, "load", BASE.fixedSpeed, BASE.fixedTemp,  
BASE.fixedMode);
```

```
S1_opt = run_sweep_opt_load(mdl, P0, loads, BASE.fixedTemp, OPT);
```

```
disp("Running Scenario 2: Speed analysis (20 points)...");
```

```
S2_base = run_sweep(mdl, P0, speeds, "speed", BASE.fixedLoad, BASE.fixedTemp,  
BASE.fixedMode);
```

```
S2_opt = run_sweep_opt_speed(mdl, P0, speeds, BASE.fixedLoad, BASE.fixedTemp,  
OPT);
```

```
disp("Running Scenario 3: Temperature analysis (20 points)...");
```

```
S3_base = run_sweep(mdl, P0, temps, "temp", BASE.fixedLoad, BASE.fixedSpeed,  
BASE.fixedMode);
```

```
S3_opt = run_sweep_opt_temp(mdl, P0, temps, BASE.fixedLoad, BASE.fixedSpeed,  
OPT);
```

```
disp("Running Scenario 4: Operator behaviour analysis (20 points)...");
```

```
S4_base = run_sweep(mdl, P0, modes, "mode", BASE.fixedLoad, BASE.fixedSpeed,  
BASE.fixedTemp);
```

```
S4_opt = run_sweep_opt_mode(mdl, P0, modes, BASE.fixedLoad, BASE.fixedSpeed,  
BASE.fixedTemp, OPT);
```

```
% ===== Save tables =====
```

```
writetable(S1_base, fullfile(projRoot,"results","tables","S1_load_base.csv"));
writetable(S1_opt, fullfile(projRoot,"results","tables","S1_load_opt.csv"));
writetable(S2_base, fullfile(projRoot,"results","tables","S2_speed_base.csv"));
writetable(S2_opt, fullfile(projRoot,"results","tables","S2_speed_opt.csv"));
writetable(S3_base, fullfile(projRoot,"results","tables","S3_temp_base.csv"));
writetable(S3_opt, fullfile(projRoot,"results","tables","S3_temp_opt.csv"));
writetable(S4_base, fullfile(projRoot,"results","tables","S4_mode_base.csv"));
writetable(S4_opt, fullfile(projRoot,"results","tables","S4_mode_opt.csv"));
```

```
% ===== Plot figures =====
```

```
plot_compare_line(S1_base.load_ratio, S1_base.E_total_kWh, S1_opt.E_total_kWh, ...
    "Load ratio", "Energy consumption per cycle (kWh)", ...
    "Effect of working load on energy consumption (baseline vs. optimized)", ...
    fullfile(projRoot,"results","figs","Effect of working load on energy consumption.png"));
```

```
plot_compare_line(S2_base.speed_ratio, S2_base.E_total_kWh, S2_opt.E_total_kWh, ...
    "Speed ratio", "Energy consumption per cycle (kWh)", ...
    "Effect of operating speed on energy consumption (baseline vs. optimized)", ...
    fullfile(projRoot,"results","figs","Effect of operating speed on energy
consumption.png"));
```

```
plot_compare_line(S3_base.T_amb_C, S3_base.E_total_kWh, S3_opt.E_total_kWh, ...
    "Ambient temperature (°C)", "Energy consumption per cycle (kWh)", ...
    "Effect of ambient temperature on energy consumption (baseline vs. optimized)", ...
    fullfile(projRoot,"results","figs","Effect of ambient temperature on energy
consumption.png"));
```

```
% For 20 operator-behaviour points, line plot is clearer than grouped bars.
```

```

plot_compare_line(S4_base.mode_op, S4_base.E_total_kWh, S4_opt.E_total_kWh, ...
    "Operator behaviour intensity (0 = smooth, 1 = aggressive)", ...
    "Energy consumption per cycle (kWh)", ...
    "Effect of operating behaviour on energy consumption (baseline vs. optimized)", ...
    fullfile(projRoot,"results","figs","Effect of operating behaviour on energy
consumption.png"));

```

```

% ===== Summary =====

```

```

Summary = summarize_results(S1_base,S1_opt,"Load");
Summary = [Summary; summarize_results(S2_base,S2_opt,"Speed")];
Summary = [Summary; summarize_results(S3_base,S3_opt,"Temp")];
Summary = [Summary; summarize_results(S4_base,S4_opt,"Mode")];

```

```

writetable(Summary,
fullfile(projRoot,"results","tables","Summary_savings_stability_efficiency_20pts.csv"));

```

```

disp("Done.");
disp("20-point figures saved to: results/figs/");
disp("20-point tables saved to: results/tables/");

```

```

%% ===== local helper functions =====

```

```

function T = run_sweep(mdl, P0, vec, kind, fixed2, fixed3, fixed4)

    rows = [];
    for i=1:numel(vec)
        switch kind
            case "load"
                load_ratio = vec(i); speed_ratio = fixed2; T_amb = fixed3; mode_op = fixed4;
            case "speed"
                load_ratio = fixed2; speed_ratio = vec(i); T_amb = fixed3; mode_op = fixed4;

```

```

    case "temp"

        load_ratio = fixed2; speed_ratio = fixed3; T_amb = vec(i); mode_op = fixed4;

    case "mode"

        load_ratio = fixed2; speed_ratio = fixed3; T_amb = fixed4; mode_op = vec(i);

    otherwise

        error("Unknown kind.");

    end

    row = run_one_case mdl, P0, load_ratio, speed_ratio, T_amb, mode_op;

    rows = [rows; row]; %#ok

end

T = struct2table(rows);

end

```

```

function Topt = run_sweep_opt_load mdl, P0, loads, fixedTemp, OPT

```

```

    rows = [];

    for i=1:numel loads

        L = loads(i);

        P = P0;

        mode_op = 0;

        if L <= OPT.low_load_threshold

            P.P_aux_W = P.P_aux_W * OPT.standby_aux_factor;

        end

        best = [];

        for s = OPT.search_speeds

            row = run_one_case mdl, P, L, s, fixedTemp, mode_op;

            if isempty(best) || row.E_total_kWh < best.E_total_kWh

```

```

        best = row;
    end
end

best.opt_note = "smooth+bestSpeed+standbyAux(lowLoad)";

rows = [rows; best]; %#ok

end

Topt = struct2table(rows);

end

function Topt = run_sweep_opt_speed(mdl, P0, speeds, fixedLoad, fixedTemp, OPT)

    mode_op = 0;

    bestS = speeds(1); bestE = inf;

    for s = OPT.search_speeds

        row = run_one_case(mdl, P0, fixedLoad, s, fixedTemp, mode_op);

        if row.E_total_kWh < bestE

            bestE = row.E_total_kWh; bestS = s;

        end

    end

end

rows = [];

for i=1:numel(speeds)

    row = run_one_case(mdl, P0, fixedLoad, bestS, fixedTemp, mode_op);

    row.speed_req = speeds(i);

    row.speed_applied = bestS;

    row.opt_note = "clampToEconomicSpeed+smooth";

    rows = [rows; row]; %#ok

end

Topt = struct2table(rows);

```

end

```
function Topt = run_sweep_opt_temp(mdl, P0, temps, fixedLoad, fixedSpeed, OPT)
```

```
    rows = [];
```

```
    for i=1:numel(temps)
```

```
        T = temps(i);
```

```
        P = P0;
```

```
        mode_op = 0;
```

```
        P.P_aux_W = P.P_aux_W + OPT.temp_ctrl_extra_W;
```

```
        P.k_eta_per_C = P.k_eta_per_C * OPT.k_eta_reduction;
```

```
        P.T_oil_init_C = P.T_ref_C;
```

```
        row = run_one_case(mdl, P, fixedLoad, fixedSpeed, T, mode_op);
```

```
        row.opt_note = "tempManagement(k_eta down)+extraAux+smooth";
```

```
        rows = [rows; row]; %#ok
```

```
    end
```

```
    Topt = struct2table(rows);
```

end

```
function Topt = run_sweep_opt_mode(mdl, P0, modes, fixedLoad, fixedSpeed,  
fixedTemp, OPT)
```

```
    rows = [];
```

```
    for i=1:numel(modes)
```

```
        reqMode = modes(i);
```

```
        appliedMode = 0; % enforce smooth/eco operation
```

```
        row = run_one_case(mdl, P0, fixedLoad, fixedSpeed, fixedTemp, appliedMode);
```

```
        row.mode_req = reqMode;
```

```

    row.mode_applied = appliedMode;

    row.opt_note = "operatorTraining(enforceSmooth)";

    rows = [rows; row]; %#ok

end

Topt = struct2table(rows);

end

function row = run_one_case(mdl, P_in, load_ratio, speed_ratio, T_amb, mode_op)

    P = P_in;

    P.load_ratio0 = load_ratio;

    P.speed_ratio0 = speed_ratio;

    P.T_amb0 = T_amb;

    P.mode_op0 = mode_op;

    assignin('base','P',P);

    simOut = sim(mdl, "ReturnWorkspaceOutputs","on");

    P_total_ts = must_get(simOut, "P_total_ts");
    P_loss_ts = must_get(simOut, "P_loss_ts");
    T_oil_ts = must_get(simOut, "T_oil_ts");

    t = P_total_ts.Time(:);
    Pt = P_total_ts.Data(:);
    Pl = P_loss_ts.Data(:);
    To = T_oil_ts.Data(:);

    row = struct();

```

```

row.load_ratio = load_ratio;
row.speed_ratio = speed_ratio;
row.T_amb_C = T_amb;
row.mode_op = mode_op;

row.E_total_kWh = trapz(t, Pt) / 3.6e6;
row.E_loss_kWh = trapz(t, Pl) / 3.6e6;
row.P_peak_kW = max(Pt) / 1000;
row.P_std_kW = std(Pt) / 1000;
row.T_end_C = To(end);

row.throughput_idx = max(speed_ratio, 1e-6);
row.E_per_throughput = row.E_total_kWh / row.throughput_idx;
row.opt_note = '';
end

```

```

function ts = must_get(simOut, name)
    try
        ts = simOut.get(name);
    catch
        error("Cannot find '%s' in simOut. Check To Workspace variable names.", name);
    end
end

```

```

function ensureDir(p)
    if ~exist(p, 'dir'), mkdir(p); end
end

```

```

function plot_compare_line(x, yBase, yOpt, xlab, ylab, titleStr, savePath)

    f = figure('Visible','on','Color','w');

    plot(x, yBase, "-o", "LineWidth", 1.7, "MarkerSize", 4.5); hold on;

    plot(x, yOpt, "-s", "LineWidth", 1.7, "MarkerSize", 4.5);

    grid on;

    xlabel(xlab);

    ylabel(ylab);

    title(titleStr);

    legend("Baseline", "Optimized", "Location", "best");

    set(gca, "FontName", "Times New Roman", "FontSize", 11, "LineWidth", 1.2);

    exportgraphics(f, savePath, "Resolution", 300);

end

```

```

function Summary = summarize_results(Tbase, Topt, tag)

    E0 = Tbase.E_total_kWh(:);

    E1 = Topt.E_total_kWh(:);

    saving = (E0 - E1) ./ max(E0, eps);

    Summary = table( ...

        string(tag), mean(E0,'omitnan'), mean(E1,'omitnan'), mean(saving,'omitnan'), ...

        mean(Tbase.P_peak_kW,'omitnan'), mean(Topt.P_peak_kW,'omitnan'), ...

        mean(Tbase.P_std_kW,'omitnan'), mean(Topt.P_std_kW,'omitnan'), ...

        mean(Tbase.E_per_throughput,'omitnan'),

        mean(Topt.E_per_throughput,'omitnan'), ...

        'VariableNames',

        {'Module','E_base_mean_kWh','E_opt_mean_kWh','Saving_mean_ratio', ...

        'Ppeak_base_mean_kW','Ppeak_opt_mean_kW', ...

        'Pstd_base_mean_kW','Pstd_opt_mean_kW', ...

        'E/Throughput_base','E/Throughput_opt'});

```

end